

AI / Forward Deploy Engineer: Take Home Assessment

Candidate Challenge

Congratulations 🎉

Only 1 out of 60 applicants make it this far. We are genuinely excited to see how you think. And, **80% of applicants that do a great challenge receive an offer.**

This challenge helps us understand how you approach real engineering problems: messy data, incomplete requirements, AI-assisted workflows, business logic, and clear communication.

We are not looking for a perfect production product. We are looking for a working, explainable solution that shows your reasoning, ownership, and ability to use AI tools responsibly.

Timebox

Please spend no more than 6 hours. Do not overbuild. A simple but correct CLI, API, or web UI is acceptable. A focused solution that you can explain well is better than an unfinished complex application.

AI Tools

AI tools are allowed, including Cursor, Claude, ChatGPT, Codex, or similar tools.

However, you are responsible for the final solution. During the live interview, you will be asked to explain your code, defend your decisions, and make a small change live.

What We Evaluate

Area	What we look for
Applied AI judgment	Can you use AI without turning the system into a black box?
Technical foundations	Can you load data, write clear logic, and produce usable output?
Business reasoning	Can you identify safe matches, risky cases, and human-review scenarios?
Challenge ownership	Can you explain, defend, and modify your solution live?
Communication	Can a finance operator or technical reviewer understand your output?

The Scenario

You are helping a finance back-office team reconcile invoices and payments.

The team receives invoices, payment records, and short operational notes from emails or internal tools. The data is messy:

- Vendor names may not match exactly.
- Some payments are partial.
- Some payments include discounts or adjustments.
- Some records have typos.
- Some payments may be duplicates.
- Some matches are safe, while others need human review.

Your Challenge

Build a small AI-assisted reconciliation assistant that helps a finance operator review invoice/payment matches.

You may use any stack you prefer. TypeScript, React, Next.js, Python, SQL, Prisma, Postgres, Supabase, or similar tools are encouraged but not required.

Required Core Version

Task	Requirement
1. Load the data	Load invoices.csv, payments.csv, and notes.json.
2. Match payments to invoices	Use your own logic with signals such as invoice ID, PO number, vendor similarity, amount, currency, reference, and notes.
3. Classify each case	Use these statuses: Matched, Partial Match, Needs Review, Unmatched, Suspicious.
4. Generate explanations	Provide a short plain-English explanation for each result. You may use an LLM, mocked AI, or AI-assisted explanation logic.
5. Show the output	Use one format: CLI/table output, JSON file, simple API, or simple web UI.
6. Add a README	Explain how to run it, your matching strategy, where AI is used, edge cases, and what you would improve.

Important

Do not use an LLM as the only decision-maker. We want to see your matching logic and judgment. Strong solutions usually keep financial decisions rule-based and use AI for explanations, note interpretation, or assisted reasoning.

Required Output Fields

- invoice ID
- matched payment ID or payment IDs
- status
- confidence score
- explanation
- remaining balance when applicable
- suggested next action

Optional Bonus

These are not required to pass the challenge. They are useful if you want to show stronger experience or extra polish.

Bonus	Description
Web UI	Simple table, filters, or detail view.
REST API	Example: GET /reconciliation.
Tests	Basic tests for matching logic and edge cases.
Database	SQLite, Postgres, Supabase, or similar persistence.
LLM integration	Real or mocked LLM with structured prompt and safe fallback.
Manual review	Approve, reject, duplicate, or resolved state.
Audit trail	Track who changed a decision and when.

Sample Data

Create these files inside your project.

invoices.csv

```
invoice_id,vendor,invoice_date,due_date,currency,amount,po_number,status
INV-1001,ACME Logistics,2026-05-01,2026-05-30,USD,1250.00,PO-8891,open
INV-1002,Grupo Norte SA,2026-05-03,2026-06-02,USD,980.50,PO-8892,open
INV-1003,Blue Ocean Supplies,2026-05-04,2026-06-04,USD,4300.00,PO-8893,open
INV-1004,ACME Logistics LLC,2026-05-08,2026-06-08,USD,700.00,PO-8894,open
INV-1005,Delta Tech Services,2026-05-10,2026-06-10,USD,1500.00,PO-8895,open
INV-1006,Northwind Foods,2026-05-11,2026-06-11,MXN,2200.00,PO-8896,open
INV-1007,Prime Retail Ops,2026-05-12,2026-06-12,USD,750.00,PO-8897,open
INV-1008,Nova Packaging,2026-05-14,2026-06-14,USD,1200.00,PO-8898,open
```

payments.csv

```
payment_id,payment_date,payer_name,currency,amount,reference
PAY-9001,2026-05-15,ACME Logistics,USD,1250.00,Payment for invoice 1001
PAY-9002,2026-05-18,Grupo Norte,USD,980.50,INV1002
PAY-9003,2026-05-20,Blue Ocean Supplies,USD,3000.00,Partial payment PO-8893
PAY-9004,2026-05-22,Unknown Vendor,USD,700.00,PO-8894
PAY-9005,2026-05-25,Delta Technology Services,USD,1490.00,Invoice 1005 discount applied
PAY-9006,2026-05-26,Northwind Food,MXN,2200.00,PO-8896
PAY-9007,2026-05-27,Prime Retail Ops,USD,750.00,Payment for INV-1007
PAY-9008,2026-05-28,Prime Retail Operations,USD,750.00,Second payment for INV-1007
PAY-9009,2026-05-29,Nova Packaging,EUR,1200.00,INV-1008
PAY-9010,2026-05-30,Random Supplier,USD,600.00,No invoice reference
```

notes.json

```
[
  {
    "source": "email",
    "text": "ACME Logistics confirmed payment for invoice INV-1001. There was a typo in the payer name from the bank export."
  },
  {
    "source": "email",
    "text": "Blue Ocean Supplies sent a partial payment for PO-8893. Remaining balance will be paid next week."
  },
  {
    "source": "slack",
    "text": "Delta Tech Services applied a 10 USD early payment discount for invoice INV-1005."
  },
  {
    "source": "ops-note",
    "text": "Prime Retail Ops may have accidentally sent the same payment twice for invoice INV-1007. Please verify before closing."
  },
  {
    "source": "email",
    "text": "Nova Packaging normally invoices in USD. Any EUR payment should be manually reviewed before reconciliation."
  }
]
```

Expected Output

Your solution does not need to match this exact structure, but it should produce equivalent information.

Example JSON output

```
[
  {
    "invoice_id": "INV-1001",
    "matched_payment_ids": ["PAY-9001"],
    "status": "Matched",
    "confidence": 0.95,
    "remaining_balance": 0,
    "suggested_action": "Auto-match",
    "explanation": "The payment likely matches the invoice. The payer name has a typo, but the amount is exact and the reference mentions invoice 1001."
  },
  {
    "invoice_id": "INV-1003",
    "matched_payment_ids": ["PAY-9003"],
    "status": "Partial Match",
    "confidence": 0.85,
    "remaining_balance": 1300,
    "suggested_action": "Keep invoice open",
    "explanation": "The payment references PO-8893 and the note confirms this is a partial payment. The remaining balance is 1300 USD."
  },
  {
    "invoice_id": "INV-1007",
    "matched_payment_ids": ["PAY-9007", "PAY-9008"],
    "status": "Suspicious",
    "confidence": 0.80,
    "remaining_balance": 0,
    "suggested_action": "Manual review required",
    "explanation": "Two payments appear to match the same invoice for the same amount. This may be a duplicate payment and should not be auto-approved."
  }
]
```

How to Submit

- Upload your code to a GitHub repository.
- Record a 5-10 minute video walkthrough using Loom, Jam, or a similar tool.
- Share both links with the recruiter: repository link and video link.

Your video should cover:

- what you built
- how to run it
- how the matching logic works
- where AI is used
- one matched example
- one partial match example
- one suspicious or needs-review example
- what you would improve with more time

Important Notes

- Please do not spend more than 6 hours.
- Do not hardcode only the expected answers without building matching logic.
- AI tools are allowed, but you must understand and explain your code.
- If using an LLM API, include safe setup instructions and do not commit secrets.
- If you mock the LLM, clearly explain the intended prompt and guardrails.
- We care more about reasoning, structure, and ownership than visual polish.
- During the live interview, you will be asked to explain your code and make a small change live.